The GAP 4 Development Manual

Administrating the GAP Project

2015

The GAP Group

The GAP Group Email: support@gap-system.org
Homepage: https://www.gap-system.org

Abstract

In this *development manual* we collect documentation of various tasks needed for the further development, maintenance and distribution of GAP.

Some sections provide practical howto's and check lists for tasks like, e.g.: committing changes, testing, wrapping archives, compiling documentation.

Other sections document certain conventions which are followed by the active members of The GAP Group. Some examples: there are several mailing lists which are used for different purposes; there is a mechanism how larger changes to the GAP system should be announced and discussed among the developers before they are committed, called *proposals*; there is a concept of *modules* where parts of the GAP system are assigned to specified maintainers.

Copyright

Contents

Chapter 1

Structure of the GAP distribution

Here is a short overview of the structure of the GAP distribution and repository.

1.1 Kernel, library, documentation, package interface, development part

...

1.2 The directories of the GAP distribution and repository

...

Chapter 2

The GAP kernel programming

2.1 Overview of the GAP kernel

The GAP kernel consists of more than 150000 lines of C code that:

- provides the run-time environment and user interface;
- interprets the GAP language;
- performs arithmetic operations with basic types of objects;
- speeds up some time-critical operations.

The GAP kernel code mainly falls into the four main categories:

1. Implementations for basic data types and structures (integers, permutations, finite field elements, etc.), which has to be in the kernel for the maximal efficiency.
2. Low level access methods for data objects (lists, ranges, records, etc.).
3. Various methods for handling complex GAP objects in component and positional representation from the kernel.
4. GAP foundations such as GAP language, garbage collector (in what follows - GC), etc.

The GAP kernel programming has four levels of sofistication:

1. The simplest form of kernel programming is to add new kernel functions for manipulating existing data types.
2. Using the "data object" type to add new binary data structures to the kernel.
3. Adding new basic (primitive) data types, such as, for example, *Floats*.
4. Modifying the foundations, for example, changing the syntax of the GAP language.

We will cover only first two levels here, while adding new basic data types or modifying the foundations are outside the scope of this manual.

2.2 "Hello World" example

On the GAP level, the kernel functionality may be accessed via kernel functions. You can recognise such functions because they will be displayed as compiled code in the GAP session:

```
gap> Display(TYPE_OBJ);
function ( obj )
    <<kernel code>> from src/objects.c:TYPE_OBJ
end
```

Let us demonstrate how to add a kernel function that will print "Hello World!". Such function has no arguments and returns nothing. To add it, we need to perform three basic steps:

1. Create the C handler, adding to the file `string.c` (this file contains the functions which mainly deal with strings) the C code doing the actual job (see the function `Pr` to print formatted output in the file `scanner.c`):

   ```
   static Obj FuncHELLO_WORLD(Obj self)
   {
       Pr("Hello World!\n", 0, 0);
       return 0;
   }
   ```

   This code should be placed somewhere in the file `string.c` before specifying `GvarFuncs`.

2. In the same file `string.c` find the list of functions to export called `GvarFuncs` and add to this list a table entry for `HELLO_WORLD`

   ```
   GVAR_FUNC_OARGS(HELLO_WORLD),
   ```

   containing respectively the string specifying the name of the function at the GAP level, the number of arguments, the string with the description of arguments, corresponding C handler and the string ("cookie") specifying the location of this handler (see definition of the structure `StructGVarFunc` in the file `system.h`).

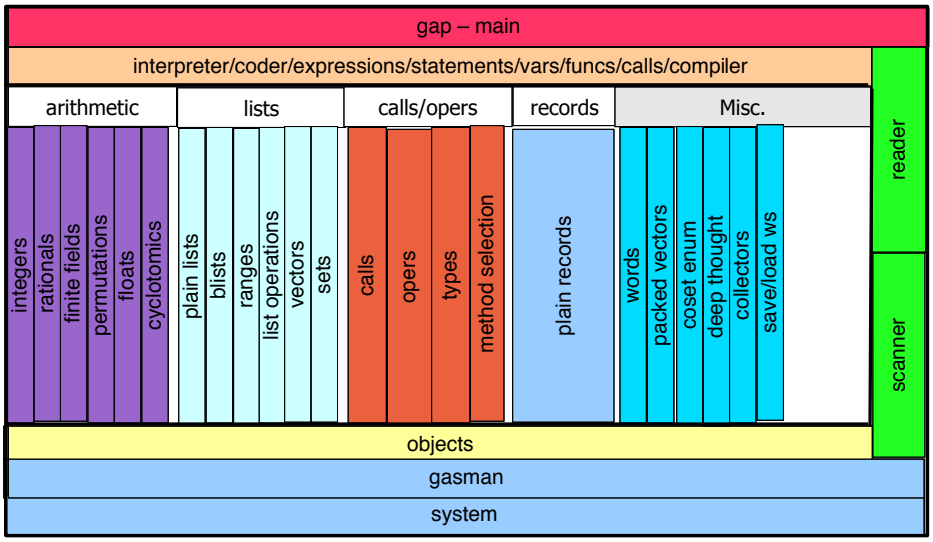
3. Compile the GAP kernel, and now `HELLO_WORLD();` will work.

   Example

   ```
   gap> HELLO_WORLD();
   Hello World!
   ```

2.3 Structure of the GAP kernel

The following picture is based on the scheme that Martin Schönert drew in 1995 and displays the structure of the GAP kernel:



The 2nd line of the scheme, the "interpreter/coder/expressions/statements/vars/funcs/calls/compiler" level is essentially the GAP runtime system. It takes in GAP code and (possibly after parsing and storing it) executes it by calling functions from the modules below. The GAP compiler is (a sort of) a drop in replacement for this, and there could be others, such as a bytecode generator/interpreter.

In the region within the thick lines the kernel code sees the same world as GAP code: it sees objects (including functions), with automatic management of memory occupied by them. The only difference is that the kernel code can look, if necessary, inside the binary content of the objects.

Unevaluated expressions or fragments of the GAP code never appear in this region and below it. Therefore, there are only a few ways to pass control back from that level:

- Calling a GAP function;
- Entering a break loop;
- Reading a file.

2.4 Garbage collection in GAP

2.4.1 GASMAN

GASMAN is the GAP memory manager. It provides an API dealing with *Bags* which are areas of memory, each with a size and a `TNUM` (type number) in the range 0–253 (type numbers are defined in the file `objects.h`; there are some spare numbers reserved for further extensions; types 254 and 255 are reserved for GASMAN. See Section 2.6 for more details).

Bags have stable handles (of C type Bag) and can be resized. When the heap is full, inaccessible bags are automatically reclaimed and live bags may be moved, but the handles don't change (handle is a pointer to a pointer to the actual data). Bag references from C local variables are found automatically, and references from C static and global variables must be declared. GASMAN recovers unreachable bags automatically – it knows that bags in C local variables *are* reachable. From the point of GC technicalities, GASMAN is generational, conservative, compacting (i.e. bags may move, but each has a permanent ID) and cooperative (i.e. it imposes certain rules on its users) memory manager.

To get the type of the bag with the identifier b, call TNUM_BAG(b).

The application specifies the type of a bag when it allocates it with NewBag and may later change it with RetypeBag (see NewBag and RetypeBag in gasman.h).

GASMAN needs to know the type of a bag so that it knows which function to call to mark all subbags of a given bag (see InitMarkFuncBags below). Apart from that GASMAN does not care at all about types.

2.4.2 GASMAN Interface

Bag is a type definition of some sort of pointer. It may be used as

```
Bag bag = NewBag(type, size);
```

Creation of a new bag may cause garbage collection, and GAP will fail if space cannot be allocated.

To reach the data in a bag, call

```
Bag *ptr = PTR_BAG(bag);
```

and keep in mind that the cost of such call is one indirection.

The *key rule* is that you must NOT hold onto this pointer during any event which might cause a garbage collection. A common hidden trap is to use

```
PTR_BAG(list)[1] = NewBag(t,s);
```

because the pointer may be evaluated before the right-hand side evaluation, and used after it. Instead of this, you should use

```
{
  Obj temp = NewBag(t,s);
  PTR_BAG(list)[1] = temp;
}
```

There is a second rule: after you assigned a bag identifier b into a bag c and before the next garbage collection, you must call CHANGED_BAG(c), unless you know that b cannot be garbage collected (e.g. it is an object like true), or c is the most recently allocated bag of all.

Other functions are:

- `RetypeBag( bag, newtnum )` changes the type of the bag with identifier `bag` to the new type `newtnum`. The identifier, the size, and also the address of the data area of the bag do not change.

- `ResizeBag( bag, newsize )` changes the size of the bag with identifier `bag` to the new size `newsize`. The identifier of the bag does not change, but the address of the data area of the bag may change. If the new size is less than the old size, GASMAN throws away any data in the bag beyond the new size. If the new size is larger than the old size, it extends the bag. Note that `ResizeBag` will perform a garbage collection if no free storage is available. During the garbage collection the addresses of the data areas of all bags may change. So you must not keep any pointers to or into the data areas of bags over calls to `ResizeBag` (see `PTR_BAG` in `gasman.h`).

- `InitMarkFuncBags( type, mark-func )`(optional) installs the function `mark-func` as marking function for bags of type `type`. The application *must* install a marking function for a type before it allocates any bag of that type.

  A marking function returns nothing. It just takes a single argument of type `Bag` and applies the function `MarkBag` to each bag identifier that appears in the bag. During garbage collection marking functions are applied to each marked bag (i.e. to all bags that are assumed to be still live), to also mark their subbags. The ability to use the correct marking function is the only use that GASMAN has for types.

  `MarkBag(b)` marks the bag `b` as live so that it is not thrown away during a garbage collection. It tests if `bag` is a valid identifier of a bag in the young bags area. If it is not, then `MarkBag` does nothing, so there is no harm in calling it for something that is not actually a bag identifier. It is important that `MarkBag` should only be called from the marking functions installed with `InitMarkFuncBags`.

  `MarkBagWeakly` is an alternative to `MarkBag`, intended to be used by the marking functions of weak pointer objects. A bag which is marked both weakly and strongly is treated as strongly marked. A bag which is only weakly marked will be recovered by garbage collection, but its identifier remains, marked in a way which can be detected by `IsWeakDeadBag`. This should always be checked before copying or using such an identifier.

  GASMAN already provides the following standard marking functions defined in `gasman.c`:

  - `MarkNoSubBags( bag )` is a marking function for types whose bags contain no identifier of other bags. It does nothing, as its name implies, and simply returns. For example, the bags for large integers contain only the digits and no identifiers of bags.
  - `MarkOneSubBags( bag )` is a marking function for types whose bags contain exactly one identifier of another bag as the first entry. It marks this subbag and returns. For example, bags for finite field elements contain exactly one bag identifier for the finite field the element belongs to.
  - `MarkTwoSubBags( bag )` is a marking function for types whose bags contain exactly two identifiers of other bags as the first and second entry such as the binary operations bags. It marks those subbags and returns. For example, bags for rational numbers contain exactly two bag identifiers for the numerator and the denominator.
  - `MarkAllSubBags( bag )` is the marking function for types whose bags contain only identifiers of other bags (for example, bags or lists contain only bag identifiers for the elements of the list or 0 if an entry has no assigned value). `MarkAllSubBags` marks every entry of such a bag. Note that `MarkAllSubBags` assumes that all identifiers are at offsets

from the address of the data area of bag that are divisible by `sizeof(Bag)`. Also, since it does not do any harm to mark something which is not actually a bag identifier, one could use `MarkAllSubBags` for all types as long as the identifiers in the data area are properly aligned as explained above (this would however slow down `CollectBags`). By default, GASMAN uses `MarkAllSubBagsDefault` which does exactly this.

2.4.3 The GASMAN Interface for Weak Pointer Objects

The key support for weak pointers is in the files `src/gasman.c` and `src/gasman.h`. This document assumes familiarity with the rest of the operation of GASMAN. A kernel type (tnum) of bags which are intended to act as weak pointers to their subobjects must meet three conditions. Firstly, the marking function installed for that tnum must use `MarkBagWeakly` for those subbags, rather than `MarkBag`. Secondly, before any access to such a subbag, it must be checked with `IsWeakDeadBag`. If that returns `true`, then the subbag has evaporated in a recent garbage collection and must not be accessed. Typically the reference to it should be removed. Thirdly, a *sweeping function* must be installed for that tnum which copies the bag, removing all references to dead weakly held subbags.

The files `src/weakptr.c` and `src/weakptr.h` use this interface to support weak pointer objects. Other objects with weak behaviour could be implemented in a similar way.

2.5 Interfaces

The modules at the bottom of the picture from the section 2.3 (*objects*, *gasman* and *system*), and white boxes (*arithmetic*, *lists*, *calls/operations*, *records*) provide interfaces intended for use in the kernel.

For example `ariths.h` provides functions like `SUM`, `PROD`, `AINV`, corresponding to the GAP operations `+`, `*` and unary `-`. These functions can be applied to any values (objects) to perform an appropriate action. Note that there is some overhead in using these general functions, if you know what your arguments are, there may be faster ways.

Another interface provides `ELM_LIST`, `ASS_LIST`, `LEN_LIST`, etc. as a general interface to any type of list.

Functions like these will even work for GAP-level objects whose arithmetic or list operations are implemented by installed methods.

Adding a kernel function is easy if it stays in the region within the thick lines, i.e. it uses interfaces from white and yellow areas, which provide kernel equivalents to the basic GAP functionality.

For example, the following kernel function will return a list containing an object, its square and its cube:

Example

```
/*  x -> [x,x^2,x^3]  */
static Obj FuncFoo1(Obj self, Obj x)
{
    Obj x2, x3;
    Obj list = NEW_PLIST( T_PLIST, 3 );
    SET_ELM_PLIST( list, 1, x );
    CHANGED_BAG( list );

    x2 = PROD( x, x );
    SET_ELM_PLIST( list, 2, x2 );
    CHANGED_BAG( list );
```

```
    x3 = PROD( x2, x );
    SET_ELM_PLIST( list, 3, x3 );
    CHANGED_BAG( list );

    SET_LEN_PLIST(list, 3);
    return list;
}
```

If speed is not of utmost importance, then it is usually better to use the ASS_LIST etc. functions instead of SET_ELM_PLIST and friends, which are more low-level and easier to misuse. For example, they automatically call CHANGED_BAG and SET_LEN_PLIST. Then the above example would become this instead:

Example

```
/*  x -> [x,x^2,x^3]  */
static Obj FuncFoo2(Obj self, Obj x)
{
    Obj x2, x3;
    Obj list = NEW_PLIST( T_PLIST, 3 );
    ASS_LIST( list, 1, x );

    x2 = PROD( x, x );
    ASS_LIST( list, 2, x2 );

    x3 = PROD( x2, x );
    ASS_LIST( list, 3, x3 );

    return list;
}
```

Finally, if all you want to do is produce a list of fixed length, you can use the NewPlistFromArgs helper function:

Example

```
/*  x -> [x,x^2,x^3]  */
Obj FuncFoo3(Obj self, Obj x)
{
    Obj x2 = PROD( x, x );
    Obj x3 = PROD( x2, x );
    return NewPlistFromArgs( x, x2, x3 );
}
```

2.6 Objects and API

The kernel representation of every GAP object is an object of C type Obj. Objects are defined in the file objects.h. Objects are actually *bags* (represented by their handles), small integers and small

finite field elements (represented by values that could not be valid handles). `Bag` is the type of bag identifiers, defined in the file `system.h`:

Example

```
typedef UInt * *        Bag;
```

Each bag is identified by its *bag identifier*, and no two live bags have the same identifier.

Note that the identifier of a bag is different from the address of the data area of the bag. This address may change during a garbage collection while the identifier of a bag never changes. Bags that contain references to other bags must always contain the identifiers of these other bags, never the addresses of the data areas of the bags.

Note that bag identifiers are recycled. That means that after a bag dies its identifier may be reused for a new bag.

0 is a valid value of the type `Bag`, but is guaranteed not to be the identifier of any bag.

The ability to distinguish between bags and other objects relies on the fact that all bag identifiers are divisible by 4.

There are lots of kernel API functions providing a uniform interface for working with objects:

- `TNUM_OBJ` (first level type), `SIZE_OBJ`, `ADDR_OBJ` (C pointer to data), `TYPE_OBJ` (full type)
- `IS_MUTABLE_OBJ`, `MakeImmutable`, `COPY_OBJ`, ...
- `PRINT_OBJ`
- `NEW_PLIST`, `SET_ELM_PLIST`, etc. for creating plain lists
- `PROD` for arithmetic (see also `SUM`, `DIFF`, etc.)

These functions are flexible: for example, `PROD` will multiply anything, but if you know what objects you will have it will be a bit faster to call the multiplication directly.

Typical implementation is a table of functions indexed by `TNUM_OBJ`. For example, this is the definition of `PROD` in the file `ariths.h` (you can safely ignore the `EXPORT_INLINE` for now):

Example

```
EXPORT_INLINE Obj PROD(Obj opL, Obj opR)
{
    UInt tnumL = TNUM_OBJ(opL);
    UInt tnumR = TNUM_OBJ(opR);
    return (*ProdFuncs[tnumL][tnumR])(opL, opR);
}
```

Additionally, `objects.h` also defines symbolic names for TNUMs.

There are lots of other API functions for strings, general lists, calling functions, etc.

2.6.1 Immediate Integers and FFEs

There are three integer types in GAP: `T_INT`, `T_INTPOS` and `T_INTNEG`. Each integer has a unique representation, e.g., an integer that can be represented as `T_INT` is never represented as `T_INTPOS` or `T_INTNEG`.

`T_INT` is the type of those integers small enough to fit into 29 bits (on 32 bit systems) respectively 61 bits (on 64 bit systems). Therefore the value range of this small integers is $-2^{28}...2^{28}-1$ respectively $-2^{60}...2^{60}-1$. Only these small integers can be used as index expression into sequences.

Small integers are represented by an immediate integer handle, containing the value instead of pointing to it, which has the format `ss<28 data bits>01` respectively `ss<60 data bits>01`.

Immediate integers handles carry the tag `T_INT`, i.e. the last bit is 1. Thus, the last bit distinguishes immediate integers from other handles which point to structures aligned on 4 byte (on 32 bit) respectively 8 byte (on 64 bit) boundaries and therefore have last bit zero. (The bit before the last is reserved as tag to allow extensions of this scheme.) Using immediates as pointers and dereferencing them gives address errors.

The first two bits `ss` are two copies of the sign bit. That is, the sign bit of immediate integers has a guard bit, that allows quick detection of overflow for addition since these two bits must always be equal.

Functions `IS_INTOBJ` and `ARE_INTOBJS` are used to test if an object (or two objects) is an (immediate) integer object. The functions `INTOBJ_INT` is used to convert a C integer to an (immediate) integer object, and `INT_INTOBJ` is used to convert the (immediate) integer object to a C integer. These functions do NOT check for overflows.

The other two integer types are `T_INTPOS` and `T_INTNEG` for positive (respectively, negative) integer values that cannot be represented by immediate integers. See comments in `integer.c` for details.

Other immediate objects are *finite field elements* (FFEs). The kernel supports small finite fields with up to 65536 elements (larger fields can be realized as polynomial domains over smaller fields). Immediate FFEs are represented in the format `<16 bit value><13 bit field ID>010`, where the least significant 3 bits of such an immediate object are always 010, flagging the object as an object of a small finite field.

The next 13 bits represent the small finite field where the element lies, and they are simply an index into a global table of small finite fields.

The most significant 16 bits represent the value of the element.

If the value is 0, then the element is the zero from the finite field. Otherwise the integer is the logarithm of this element with respect to a fixed generator of the multiplicative group of the finite field plus one. In the following descriptions we denote this generator always with z, it is an element of order $ord-1$, where *ord* is the order of the finite field. Thus 1 corresponds to $z^{1-1}=z^0=1$, i.e., the one from the field. Likewise 2 corresponds to $z^{2-1}=z^1=z$, i.e., the root itself.

This representation makes multiplication very easy, we only have to add the values and subtract 1 , because $z^{a-1}*z^{b-1}=z^{(a+b-1)-1}$. Addition is reduced to multiplication by the formula $z^a+z^b=z^b*(z^{a-b}+1)$. This makes it necessary to know the successor z^a+1 of every value.

The finite field bag contains the successor for every nonzero value, i.e., `SUCC_FF(<ff>)[<a>]` is the successor of the element `<a>`, i.e, it is the logarithm of $z^{a-1}+1$. This list is usually called the Zech-Logarithm table. The zeroth entry in the finite field bag is the order of the finite field minus one.

`T_INT` and `T_FFE` are reserved `TNUM`s for immediate integers and FFEs. `TNUM_OBJ` produces them if needed and otherwise calls `TNUM_BAG`.

The kernel design allows that other immediate types could be added in the future.

2.6.2 Arithmetics

Arithmetic operations package is implemented in files `ariths.h` and `ariths.c`. In particular, it defines functions like `SUM(obj1, obj2)`, `DIFF`, `PROD` etc., which accept (and may return) objects of

any kind, including immediate objects. Selection of the appropriate method is handled via calling TNUM_OBJ for arguments and then calling the appropriate method from the table of functions, for example:

```
EXPORT_INLINE Obj SUM(Obj opL, Obj opR)
{
    UInt tnumL = TNUM_OBJ(opL);
    UInt tnumR = TNUM_OBJ(opR);
    return (*SumFuncs[tnumL][tnumR])(opL, opR);
}
```

The default entry of the table of functions just calls back to the GAP level to look for the installed methods (see e.g. SumObject in ariths.c), but note that kernel methods installed in tables *OVERRIDE* GAP installed methods.

If you expect to handle mainly small integers, then it is significantly faster to do:

```
if ( ! ARE_INTOBJS( <opL>, <opR> ) || ! SUM_INTOBJS( <res>, <opL>, <opR> ) )
    <res> = SUM( <opL>, <opR> );
```

instead of <res> = SUM(<opL>, <opR>), where SUM_INTOBJS is a function, which returns 0 if the answer overflows and a large integer needs to be created.

Finally, note that lots of functions in ariths.h called C_<something> are used mainly by the compiler.

2.6.3 Functions

The function call mechanism package is implemented in files calls.h and calls.c. A function object in GAP is represented by a bag of the type T_FUNCTION. The bag for the function f contains eight pointers fo C functions - its handlers. These eight functions are for 0,1,2,...,6 arguments and X arguments respectively, and the *i*-th handler can be accessed using the function HDLR_FUNC(f, i). This is exactly what is done by functions CALL_0ARGS, CALL_1ARGS, ..., CALL_6ARGS and CALL_XARGS, which simply call the appropriate handlers, passing the function bag and the arguments. CALL_0ARGS is for calls passing no arguments, CALL_1ARGS is for calls passing one argument, and so on. Thus, the kernel equivalent of the GAP code r := f(a,b) is r = CALL_2ARGS(f,a,b). CALL_XARGS is for calls passing more than 6 arguments or for variadic functions, and it requires the arguments to be collected in a single plain list.

There is a range of standard handlers that deal with calls to regular GAP functions, to operations, attributes and properties and that also deal with GAP variadic functions given as function(arg).

For kernel functions, the handler is the actual function which does the work. A typical handler (for 1-argument function) looks like

```
Obj FuncLength(Obj self, Obj list)
{
    return INTOBJ_INT(LEN_LIST(list));
```

```
}
```

Often the handler has to do some checks on arguments as well. In the example above LEN_LIST takes care of this (mainly because the jump table for LEN_LIST contains error functions for non-lists). Every handler must be registered (once) with a unique "cookie" by calling InitHandlerFunc(handler, cookie) before it is installed in any function bag. This is needed so that it can be identified when loading a saved workspace. *cookie* should be a unique C string, identifying the handler.

To create a function object, there are three functions NewFunction, NewFunctionC, and NewFunctionT, defined in the file calls.c.

NewFunction(name, narg, nams, hdlr) creates and returns a new function. *name* must be a GAP string containing the name of the function. *narg* must be the number of arguments, where -1 indicates a variable number of arguments. *nams* must be a GAP list containing the names of the arguments. *hdlr* must be the C function (accepting *self* and the *narg* arguments) that will be called to execute the function.

NewFunctionC does the same as NewFunction, but expects *name* and *nams* as C strings. NewFunctionT also does the same as NewFunction, but has two extra arguments that allow to specify the *type* and *size* of the newly created bag.

For example, you can make a function object using the code like this:

```
lenfunc = NewFunctionC("Length", 1, "list", FuncLength);
```

2.6.4 Lists

The GAP kernel provides a generic list interface, which is equivalent to using list[i] etc. in the library. This interface works for all types of lists known to GAP, including virtual lists. It has functions like IS_LIST (returns C Boolean), LEN_LIST (returns C integer), ISB_LIST, ELM_LIST, ELMO_LIST(<list>, <pos>) (returns 0 if <list> has no assigned object at position <pos>) and ASS_LIST, defined in the file lists.h Implementation of these functions is done via tables of functions indexed by TNUM.

Nevertheless, it will be not much faster to write your C code using such functions, for example, to reverse an arbitrary list, than to do the same working in GAP. However, in case of *plain lists* coding in C actually ought to produce some speedup.

A plain list is a list that may have holes and may contain elements of arbitrary types. A plain list may also have room for elements beyond its current logical length. The last position to which an element can be assigned without resizing the plain list is called the physical length.

If you need to create a plain list, use NEW_PLIST(<tnum>,<plength>), where <tnum> should be T_PLIST_<something>, and <plength> is physical length in bags. Furthermore, LEN_PLIST gets its logical length, and SET_LEN_PLIST sets its logical length. Note that ELM_PLIST is faster than ELM_LIST.

2.6.5 Data objects

Positional and *Component* objects are made from lists and records using Objectify. They contain their *Type* and data accessible with ![] and !. operations.

Data objects also contain their *Type*, but the data is only accessible via kernel functions. Data can be anything you like *except* bag references. The garbage collector doesn't see inside them. At a minimum, construction and basic access functions need to be written in the kernel. For example, compressed vectors are done this way.

2.7 Rules of Kernel Programming

2.7.1 Three golden rules

Three golden rules of GAP kernel programming:

1. Real C pointers into objects (returned by `ADDR_OBJ`) must *not* be held across anything that could cause a garbage collection (*GC*).
2. If you add a new object to another one (e.g. put it in a list) you must call `CHANGED_BAG` on the container, otherwise the new object may get lost in a GC.
3. Don't use malloc: actually using it a little bit is usually safe, and it's safe if you don't ever want to expand the GAP workspace.

2.7.2 Common kernel traps

More things can cause a garbage collection than you expect:

- printing (might be to string stream)
- generic list or record access (might be handled by GAP methods)
- integer arithmetic (might overflow to large integers)

Be careful of things like

Example

```
ELM_PLIST(l, 3) = something that might cause GC
```

This expands in C to `*((*l)+3) = something`. The compiler is allowed to follow the inner *, then evaluate the right-hand side, then the outer *. This will be broken by the garbage collection.

2.7.3 One More Bit of GASMAN interface

When you store a Bag ID in a C global variable, you must declare the address of the global to GASMAN (so the GC knows that the bag is alive). This is done by calling `InitGlobalBag` passing the address and another "cookie" which is used by save/load.

There are nice ways to do all the global and handler initializations from tables.

2.8 The GAP compiler

2.8.1 Compiling GAP Code

The GAP compiler converts GAP code into kernel functions. The compiled code then can be loaded into a running kernel (on UNIX or Mac OS). The resulting code still has to do lots of checks, so usually it will not be as fast as hand-written C program. The performance gain is significant for code that spends a lot of time in loops, small integer arithmetic, etc., and will be not significant if code spends most of its time in the kernel or elsewhere in library.

In the following example we compile the file `foo.g` and then load it during the subsequent GAP session:

Example

```
$ cat > foo.g
foo := x -> [x,x^2,x^3];
$ gac -d -C foo.g
... compilation to C file ...
$ gac -d foo.g
... compilation to .so file ...
$ ls -l foo.so
-rwxr-xr-x 1 sal 158 4999 2007-09-11 10:19 foo.so*
$ gap -b
GAP4, Version: 4.dev of today, ....
gap> LoadDynamicModule("./foo.so");
gap> Print(foo);
function ( x )
    <<compiled GAP code>> from foo.g:1
end
gap> foo(3);
[ 3, 9, 27 ]
```

The compiler code will look as follows:

Example

```
/* return [ x, x ^ 2, x ^ 3 ]; */
t_1 = NEW_PLIST( T_PLIST, 3 );
SET_LEN_PLIST( t_1, 3 );
SET_ELM_PLIST( t_1, 1, a_x );
CHANGED_BAG( t_1 );
t_2 = POW( a_x, INTOBJ_INT(2) );
SET_ELM_PLIST( t_1, 2, t_2 );
CHANGED_BAG( t_1 );
t_2 = POW( a_x, INTOBJ_INT(3) );
SET_ELM_PLIST( t_1, 3, t_2 );
CHANGED_BAG( t_1 );
SWITCH_TO_OLD_FRAME(oldFrame);
return t_1;
```

Note that the original GAP code (more or less) appears as comments.

If you want to see or modify the intermediate C code, you can also instruct the compiler to produce only the C files by using the option `-C` instead of `-d`.

There are some known problems with C code produced with the GAP compiler on 32 bit architectures and used on 64 bit architectures (and vice versa).

There are more ways to exploit the GAP compiler apart from just compiling the GAP code:

- You can compile your code and then hand-optimize critical sections. For example, you can replace calls to `POW` by calls to `PROD` or even to the product function for particular kinds of objects. You must be aware of a risk of a segmentation fault if you will call your function with wrong arguments.
- You can use the compiler to generate a shell that can be dynamically loaded, and then fill in your own C code in this shell. This approach is used in Browse, EDIM and IO packages. In this case you must make it sure that your code complies with rules.

2.8.2 Suitability for Compilation

Typically algorithms spend large parts of their runtime only in small parts of the code. The design of GAP reflects this situation with kernel methods for many time critical calculations such as matrix or permutation arithmetic.

Compiling an algorithm whose time critical parts are already in the kernel of course will give disappointing results: Compilation will only speed up the parts that are not already in the kernel and if they make us a small part of the runtime, the overall gain is small.

Routines that benefit from compilation are those which do extensive operations with basic data types, such as lists or small integers.

2.8.3 Compiling Library Code

This subsection describes the mechanism used to make GAP recognize compiled versions of library files. Note that there is no point in compiling the whole library as typically only few functions benefit from compilation as described in Section 2.8.2.

All library files that come with GAP (not the files that belong to GAP packages) are read using the internal function `READ_GAP_ROOT`. This function then checks whether a compiled version of the file exists and if its CRC number (see (**Reference: CrcFile**)) matches the file. If it does, the compiled version is loaded. Otherwise the file is read. You can start GAP with the `-D -N` option to see information printed about this process.

To make GAP find the compiled versions, they must be put in the `bin/`*`systemname`*`/compiled` directory (*`systemname`* is the name you gave for compilation, for example `i386-ibm-linux-gcc2`). They have to be called according to the following scheme: Suppose the file is `humpty/dumpty.gi` in the GAP home directory. Then the compiled version will be `bin/`*`systemname`*`/compiled/humpty/gi/dumpty.so`. That is, the directory hierarchy is mirrored under the `compiled` directory. A further directory level is added for the suffix of the file, and the suffix of the compiled version of the file is set to `.so` (as produced by the compiler).

For example we show how to compile the `combinat.gi` file on a Linux machine. Suppose we are in the home directory of the gap distribution.

Example

```
bin/i386-ibm-linux-gcc2/gac -d lib/combinat.gi
```

creates a file `combinat.so`. We now put it in the right place, creating also the necessary directories:

Example

```
mkdir bin/i386-ibm-linux-gcc2/compiled
mkdir bin/i386-ibm-linux-gcc2/compiled/lib
mkdir bin/i386-ibm-linux-gcc2/compiled/lib/gi
mv combinat.so bin/i386-ibm-linux-gcc2/compiled/lib/gi
```

If you now start GAP and look, for example, at the function `Binomial` (**Reference: Binomial**), defined in `combinat.gi`, you see it is indeed compiled:

Example

```
gap> Display(GaussianCoefficient);
function ( n, k, q )
    <<compiled GAP code>> from GAPROOT/lib/combinat.g:26
end
```

The command line option `-M` disables the loading of compiled modules and always reads code from the library.

2.9 Global Variables

2.9.1 Global variables

The part of the kernel that manages global variables, i.e., the global namespace, is contained in the files `gvars.h` and `gvars.c`. Global variables have an internal form, of type `GVar` (actually an index into a hash table). They may be obtained with `GVar gv = GVarName(<string>);`. A global variable may be safely held on to across GC (but not across save/load workspace).

To manipulate with global variables, you may use `AssGVar(gv, <obj>)` or `ValGVar( gv )` (returns 0 if unbound, not an error), etc.

2.9.2 Tracking global variables

The C code

```
Obj Stuff;
InitCopyGVar( "stuff", &Stuff );
```

causes `Stuff` to track the value of the global named "stuff". When the global is changed, the new value will be put in the C variable.

A useful refinement is `InitFopyGVar`. This does the same, provided that the value assigned is a function, otherwise the C variable points to a function that prints a suitable error message.

These are usually set up during initialization of a kernel module.

2.10 The Kernel Module Structure and interface

2.10.1 The Kernel Module Structure

Apart from a very small amount of glue, all of the kernel (including compiled GAP code) is organised into modules.

Basically each module consists of one `.c` file and one `.h` file. The file `gap.c` contains the variable `InitFuncsBuiltinModules` which is initialized with a list of functions such as `InitInfoInt`. Each such function, when called, returns a data structure describing a module, which typically looks as follows:

```
/*************************************************************************
**
*F  InitInfoInt() . . . . . . . . . . . . . . . . . table of init functions
*/
static StructInitInfo module = {
    .type = MODULE_BUILTIN,
    .name = "integer",
    .initKernel = InitKernel,
    .initLibrary = InitLibrary,
};
```

The first part is just information, but the functions from `initKernel` down are the key to the interface to kernel modules.

2.10.2 The Kernel Module Interface

The general rule is that if a 0 appears, the function is skipped; indeed, you may omit entries which are 0.

When GAP is starting up, it does the following:

- Calls the `initKernel` functions of all those modules that have one. If any of them return non-zero then it aborts.
- Then calls the `initLibrary` functions likewise
- Then the `checkInit` functions

The other three functions relate to save/load workspace:

- `preSave` functions are called before saving.
- `postSave` functions (in the reverse order) after saving
- `postRestore` functions are called after a workspace has been loaded to finish linking up the kernel and workspace.

2.10.3 Sequences of Events

Normal GAP startup (no -L)

- All `initKernel` methods called (abort on non-zero return)
- All `initLibrary` methods called (abort on non-zero return)
- All `checkInit` methods called (abort on non-zero return)
- `init.g` read

Startup loading a workspace

- All `initKernel` methods called (abort on non-zero return)
- Workspace loaded, global bags and handlers linked up
- All `postRestore` methods called (abort on non-zero return)
- Enter the main loop

2.10.4 Saving a Workspace

- All `preSave` methods called (recover on non-zero return)
- Full garbage collection
- After this, workspace must be "clean" – no non-objects stored where objects are expected (eg in plain lists)
- Saved workspace file written
- All `postSave` methods called in reverse order

2.10.5 Excerpts from InitKernel from integer.c

```
/****************************************************************************
**
*V  GVarFuncs . . . . . . . . . . . . . . . . . . list of functions to export
*/
static StructGVarFunc GVarFuncs [] = {

    GVAR_FUNC_2ARGS(QUO_INT, a, b),
    GVAR_FUNC_1ARGS(ABS_INT, n),
    . . .
    { 0 }
};

/****************************************************************************
**
*F  InitKernel( <module> )  . . . . . . . . . initialise kernel data structures
*/
```

```
static Int InitKernel (
    StructInitInfo *    module )
{
    UInt                t1,  t2;

    /* init filters and functions                                          */
    InitHdlrFiltsFromTable( GVarFilts );
    InitHdlrFuncsFromTable( GVarFuncs );
    . . .
```

2.10.6 Commentary

- The items in the array of `StructGVarFunc` objects define the GAP-callable global functions exported by the module
- `InitHdlrFuncsFromTable` initializes all the handlers with the given cookies
- In the `initLibrary` function of the same model, we see this:

```
/****************************************************************************
**
*F  InitLibrary( <module> ) . . . . . .  initialise library data structures
*/
static Int InitLibrary (
    StructInitInfo *    module )
{
    /* init filters and functions                                          */
    InitGVarFiltsFromTable( GVarFilts );
    InitGVarFuncsFromTable( GVarFuncs );
    . . .
```

- This call `InitGVarFuncsFromTable` creates the actual function objects (in the workspace) using `NewFunctionC`
- It also installs these objects in global variables and makes those variables read-only

2.10.7 Other Similar Functions

- There are other support functions for module initialization
- Defined in `gap.c`
- `InitHdlr<xxxx>FromTable` and `InitGVar<xxxx>FromTable` exist for filters, attributes, properties and operations
- Structure definitions are found in `system.h`

2.10.8 Importing from the Library

- There are data structures (e.g. types) and functions (e.g. for method selection) that are used by the kernel, but easier written in GAP
- These are mainly in `.g` files
- They are imported into the kernel using `ImportGVarFromLibrary` and `ImportFuncFromLibrary`
- These are basically just `InitCopyGVar` and `InitFopyGVar`
- They also make the GVars read-only and keep some records
- At the end of `read1.g` the GAP function `ExportToKernelFinished` is called
- This checks that all the imported GVars have actually been read.
- Until this is done, some functionality may not be usable yet (e.g. `TYPE_OBJ`).

2.10.9 InitKernel

Typical things to do in `initKernel` functions:

- Call `InitHdlrFuncsFromTable`
- Install names for bag types in `InfoBags`
- Install marking functions for bag types (needed for garbage collection)
- Install functions in kernel tables for...
  - Saving
  - Loading
  - Printing
  - Arithmetic operations
  - Type determination
  - Copying (and MakeImmutable)
  - List access, length, and associated functions
- Initialize GVar copies and fopies and imports
- Add entries to various tables used by the list machinery
- DO NOT create ANY bags in the workspace (they would be lost in a restore workspace)

2.10.10 initLibrary and postRestore

- These functions can create or access structure in the workspace
- Often `initLibrary` also calls `postRestore`
- Typical `postRestore` activities:
  - running `GVarName` to get the numbers of global variables of interest
  - likewise `RNamName` to get the numbers for record field names
  - getting the length of things like the list of GVars into kernel variables
- other `InitLibrary` activities:
  - Call `InitGVarFuncsFromTable` and its friends, using the same tables that were passed to the `InitHdlr` functions
  - Create any other global variables (not holding functions)
  - Create any special functions (eg ones that have handlers for several ariths)
- Allocate any scratch areas needed in the workspace

Chapter 3

Maintaining the GAP kernel and library

3.1 Debugging the GAP Kernel

The GAP kernel supports a variety of options which can be enabled by running `configure` which provide various checks which can be useful when writing and debugging GAP kernel code.

The first option, `-enable-debug`, is well supported and can be used whenever the GAP kernel, or kernel modules, are being developed (although developers should still check their code works without it).

`-enable-debug` enables assertions throughout the kernel. These assertions are implemented using the `GAP_ASSERT` macro. These assertions are disabled when GAP is compiled without `-enable-debug`. When `-enable-debug` is enabled, `KernelDebug` will be shown in the line starting `Configuration:` when GAP is started.

The next two options are more experimental, and are not designed to be constantly enabled. They are particularly designed to track down bugs relating to memory corruption relating to GASMAN (GAP's memory manager). They cannot both be enabled at the same time. These options assume you are familiar somewhat familiar with the internals of GASMAN (see gasman.c for more information). In particular, GASMAN represents memory using an object of type Bag. The contents of these Bags can be moved during garbage collection.

`-enable-memory-checking` makes GAP check for C pointers to the content of Bags being used after a new Bag has been created or a Bag is resized. GASMAN moves Bags during garbage collection, which can happen whenever a new Bag is created or a Bag is resized. However, as it is very unlikely that any single allocation will cause a garbage collection, these bugs trigger very rarely. Also the problems caused by these bugs are, if the C pointer is written to, a random other Bag, somewhere in GAP, is changed.

After configuring, the memory checking must still be turned on. This can be done either by passing `-enableMemCheck` to GAP's command line, or calling `GASMAN_MEM_CHECK(1)`. Note that enabling these tests makes GAP VERY, VERY slow. It can (depending on the machine and operating system) take GAP over a day to start, and load all standard packages. The recommending way to use this option is to start GAP, and then load small test files to try to isolate the problem. `MemCheck` will be shown in the line starting `Configuration:` when GAP is started.

Known bugs: GAP will crash if `IO_fork` from the IO package is called while memory checking is enabled.

`-enable-valgrind` makes changes to GASMAN so it is compatible with the valgrind memory checking program. Without this, Valgrind will report many incorrect warnings relating to GASMAN

and no useful warnings.

At present, this `-enable-valgrind` only checks for invalid writes to the last bag which was created. Also, this option does not do anything unless GAP is run through Valgrind (for example by running `valgrind gap`.

Modules, PROPOSALs, ...

3.2 ...

...

Chapter 4

Maintaining the GAP documentation

This chapter explains shortly how the GAP documentation is organized and how to produce the viewable documents from the sources.

4.1 The source of the GAP documentation

Mention that manual examples should fit in 72 character width, and that examples should run as shown if all examples within a chapter are run in a fresh GAP in default configuration with `ScreenSize([72]);`.

4.2 How to compile the GAP manuals from the sources

To compile the GAP manuals from sources, call

```
make doc
```

in the GAP root directory. This will compile each of the three main GAP manuals twice to ensure that cross-references are resolved.

If you need to compile a single manual book, call

```
gap makedocrel.g
```

in the corresponding directory (`ref`, `tut`, `hpc` or `dev`). This is convenient when you're working on the documentation and want to check if the changed manual book compiles, so you need not to recompile all manual books from scratch.

Chapter 5

Testing GAP

TODO: This chapter is partially outdated and has to be revised

In this chapter we describe several tests which should be applied regularly to the development and release candidate versions of GAP in the hope to reduce the number of bugs and potential other problems (such as conflicts between the main GAP system and packages) in the released version.

These tests cover GAP computations, in the sense that actual output is compared with expected output, as well as formal aspects of the documentation.

5.1 `.tst`-files

The directory `tst` in the GAP root directory contains directories, each of which contains files whose names have the suffix `.tst`. The format of each of these files must be suitable for being read with the function `Test`, see section (**Reference: Test Files**). If functionality from GAP packages is needed in a `tst` file then it is recommended to execute these tests only if the packages are really available, and to load the packages explicitly, for example as follows.

```
gap> if LoadPackage( "ctbllib" ) <> fail then
>      if Irr( CharacterTable( "WeylD", 4 ) )[1] <>
>           [ 3, -1, 3, -1, 1, -1, 3, -1, -1, 0, 0, -1, 1 ] then
>        Print( "problem with Irr( CharacterTable( \"WeylD\", 4 ) )[1]\n" );
>      fi;
>    fi;
```

Moreover, it is recommended to group tests that require packages to be performed after all other tests are completed. The directory `tst/testinstall` contains a subset of tests which are recommended after installing GAP, see `https://www.gap-system.org/Download/INSTALL`. The idea is that the examples in these `.tst` files are typical GAP computations, and that running these tests requires only a few minutes. The GAP script `tst/testinstall.g` will run the tests in `tst/testinstall`.

Further tests are contained in `tst/teststandard`. The script `tst/teststandard.g` will run all tests in both `tst/teststandard` and `tst/testinstall`.

5.2 Checking examples in the main manuals

The reproducibility of the output of examples in the main GAP manuals (the two manuals in the directories `ref` and `tut`) is promised in section (**Tutorial: Starting and Leaving GAP**), under the

condition that all examples of a manual chapter are run immediately after starting GAP. Section (**GAPDoc: Testing Manual Examples**) contains another paragraph about the use of manual examples for testing purposes.

There is no guarantee that package manuals behave this way. It is up to the package authors whether they want to promise something similar for their package.

5.3 Tests for packages

Besides tests of the functionality of packages, we are also interested in the effects of packages on the functionality of the main system. The output that is shown in examples in the GAP manuals as well as the output that is prescribed in the test files in `tst` should be the same when GAP is run without packages or with all packages available. Therefore, the tests in the standard test suite (see 5.6) are run several times, in a GAP that loads no packages and in a GAP that loads all available packages and/or all available autoloaded packages.

Concerning tests of package functionality, in principle the package authors are responsible for providing and running tests for their packages. However, it makes sense to provide test files analogous to those described in section 5.1.

If the record in the `PackageInfo.g` file of the package (see (**Reference: The PackageInfo.g File**)) contains the component `TestFile` then the file given by the value is read as a part of the regular GAP test suite, see section 5.6. So this file should contain `Test` statements for suitable files that are part of the package distribution. Since these tests are intended to be run regularly, they should require not more than a few hours of CPU time. (Of course also longer examples can make sense but they should not be listed in the file given by the `TestFile` component.)

Note that the inclusion of the above package tests refers to the latest released version of each package, not to its development version.

Since the format of the documentation for a package is not prescribed, there is no generic tool for checking manual examples of packages. One possibility to include the examples from package manuals in the standard tests is to create a `.tst` file with these examples from the manual. (For one implementation of this approach, see the script `etc/makexpl` in the `ctbllib` package. It produces a file `tst/docxpl.tst` in the package directory, containing all examples from the package manual.)

5.4 Checking references and cross-references of manuals

This is still to be provided (and then documented here). Perhaps the easiest way is a check of cross-references in and between the HTML versions of the manuals; and perhaps this should be viewed as a part of the more general task to check the validity of (in particular cross-references in) the GAP website.

(Possible ingedients are `doc/test/CheckBooks.g` and `dev/LinksOfAllHelpSections.g`.)

For the manuals in the formats given by the GAPDoc package, multiply defined labels and non-resolved references will be reported at the manual build stage.

5.5 Detecting potential naming conflicts

Also this is still to be provided (and then documented here).

5.6 The standard test suite

The standard test suite is given by several targets in the `Makefile` in the GAP root directory. (So the file `Makefile.in` is relevant for the maintenance.) It consists of

- The tests contained in `tst/testinstall`, see 5.1,
- All tests in all subdirectories of `tst`, see 5.1,
- the tests of the main manuals described in section 5.2,
- the package tests described by the `TestFile` components of `PackageInfo.g` files of all accepted or deposited packages, and
- the tests of packages loading with `LoadPackage` (**Reference: LoadPackage**) and `LoadAllPackages` (**Reference: LoadAllPackages**)

As is stated in Section 5.3, each test is run at least twice, once in a GAP that loads no packages and then in a GAP that loads all available packages and/or all available autoloaded packages.

Before running these tests, you should make sure that the system is up to date, in particular,

- compile package executables where applicable,
- call `make doc` in the root directory, in order to process the main documentation,

Then proceed as follows in the GAP root directory.

- Call `make testinstall`. This reads the file `tst/testinstall.g`, which should require just a few minutes.
- Call `make testmanuals`. This runs the examples in the main GAP manuals, and should also require only a few minutes. (Note that a new GAP process is started for each manual chapter. In order to accelerate this, a GAP workspace is created in the beginning, is reused for each chapter, and removed in the end.)
- Call `make teststandard`. This runs the tests in `tst`, and may require a couple of hours.
- Call `make testpackages`. This starts a new GAP session for each accepted or deposited package, and reads the file given by the component `TestFile` in the `PackageInfo.g` file (if this is available). It may require several hours.
- Call `make testpackagesloading`. This loads each accepted or deposited package in a new GAP session, then starts a new GAP session and calls `LoadAllPackages` (**Reference: LoadAllPackages**) to check that all packages may be loaded simultaneosuly. It require just a few minutes.

In each case, GAP is called with the command line options `-m 100m -o 550m -A -N -q -x 80 -r`, cf. (**Reference: Command Line Options**), and the assertion level is set to 2 before the tests.

The output is written to files in the directory `dev/log`, which does not belong to the distribution of GAP. The file names start with the name of the target (`testinstall`, `teststandard`, etc.), followed by `1` for the case that no packages are loaded before the tests, by `2` if all packages are loaded before the tests, and by `A` if only autoloaded packages are loaded before the tests, followed by the date and

time when the test was started, in the format `_YY_MM_DD_HH_MM`. For the package tests, the output for each package in question is written to a file of its own, and the package name is appended to the file name.

Note that in the case that no package is loaded initially, some packages may be loaded during the tests, due to `LoadPackage` (**Reference: LoadPackage**) statements in the test files. So a list of all packages that are loaded at the end of the tests can be found at the end of the output files.

5.7 Test evaluation

After the tests, please inspect the output files and deal with the differences that were found. The following situations may be expected.

- There are differences between the prescribed output and the actual output. This may be the result of code improvements and thus intentional; then the prescribed output should be replaced by the actual one. It may be a side-effect of code changes, for example new or changed methods may choose different generators for a group or a different ordering of conjugacy classes; then one should make sure that the changed output is not caused by worse methods being used than before. If the prescribed and the actual value are essentially different (e.g., `true` vs. `false` or different numerical values) then probably we have a serious problem n the sense that (at least) one of the two values is wrong.

- There are differences between the results of a test without packages and the same test with packages loaded. It may be that a package provides a better solution than the one that was expected when the input example had been prepared; in this case, it is perhaps advisable to ask the maintainers of the relevant code to modify the example in question, since it is apparently no longer typical. It may also be that a new package version breaks functionality; in this case, the package authors should be informed.

- A more subtle difference is a change in the runtime, which is listed more or less detailed in the output files. If the runtime increases w.r.t. an earlier GAP version then this should be reported to the GAP group. If the runtime with packages is substantially bigger than without packages then the package authors should be informed.

When test input is adjusted to the current behaviour, it should be adjusted for the situation that no packages are loaded. (In the case of test input for a package, this means that only the needed packages are loaded, see the component `Dependencies.NeededOtherPackages` in the file `PackageInfo.g`.) Note that testing the changed behaviour introduced by another package would be an issue for tests that belong to this package.

5.8 Test Issues for Releases

The following additional issues are related to testing when a new version of GAP is to be released.

- Run all tests.

5.9 Open items concerning tests

- Add a paragraph about general tests for features: When new documentation is added, add meaningful examples to this documentation, from which users get an idea what typical uses are. In a corresponding file in the `tst` directory, add examples which cover also pathological input such as empty lists. Add also interesting computations to the tests which are perhaps to be excluded from the regular testing; for features in a package, perhaps distinguish between the tests that are covered by the `TestFile` component of the `PackageInfo.g` file and hard tests.

Chapter 6

Version control using Git

6.1 Introduction

6.1.1 Using Git

...

Chapter 7

Distributing packages via the GAP sites

This chapter describes how to collect information on packages and to maintain their re-distribution by The GAP Group.

7.1 ...

[Transfer existing READMEs to Manual format]

Chapter 8

Maintaining the GAP website

This chapter describes how the information accessible on www.gap-system.org is stored and collected, and how it is transformed into web pages.

8.1 Overview

The GAP website (in the following just called "website") has a tree structure for easier navigation and overview. Each node and each leaf of the tree is a web page. Every single page resides somewhere in this tree. This position is shown in the navigation bar on the left hand side, and the user can navigate through the tree using this navigation bar. However, pages can still link to other pages that reside in some other branch of the tree.

All the sources for the web pages are kept in the git repository `https://github.com/gap-system/GapWWW`. So you can clone this repository using

```
git clone https://github.com/gap-system/GapWWW
```

The web server in St Andrews also uses its clone, updates it to the latest revision of the `master` branch, runs the Jekyll tool and then serves the pages. Another named branch is called `testing` and it is served on the password protected version of the GAP website at `https://test.gap-system.org` where work in progress may be published to be reviewed internally.

The GAP website has some pages that are treated specially such as the GAP manuals, the pages for the GAP packages, the pages for the GAP bibliography, and the (old) GAP forum archive. The setup for these special pages is described in Sections 8.6 to 8.8 in this chapter.

8.2 Getting started

There are several possible workflows dependently on how much efforts you would like to commit to the website maintenance.

A minimalistic scenario for small improvements (e.g. correcting details and fixing typos) only requires to install git and then:

1. Clone the Website repository: `git clone https://github.com/gap-system/GapWWW`
2. Make changes in a feature branch

3. Submit your changes as GitHub pull request so that website administrator(s) can review and merge this update.

If you are one of website administrator(s), then you will also need to be able to access the web server in St Andrews via `ssh` to run certain update scripts and copy necessary data.

8.3 Git usage

We assume here that you are familiar with the standard git commands `git clone`, `git pull`, `git push`, `git update`, `git commit` etc.

The source files for the web site are kept in the git repository `https://github.com/gap-system/GapWWW`. You may clone it by doing

```
git clone https://github.com/gap-system/GapWWW
```

This command creates in your current directory a directory `GapWWW` with the complete source tree of the web site.

Source files are treated like any other source file in the git repository, that is you can update, modify, commit, add, remove them as usual.

The only thing one has to understand with respect to git is which implications the branch in which the change has appeared will have on the process of its publication:

- Changes in the `master` branch will not be automatically published on the web server. They will be reviewed by the website administrator who will then have to run the update script on the server in St Andrews as described in Section 8.4 to make them available online.
- Changes in feature branches (which you may create to keep some work in progress) will not be visible anywhere.

A little comment on the rationale behind this setup might be in order. It allows that more than one person works independently on the website and those people exchange versions via git, without publishing them immediately. The actual guidelines who does what in this process should be agreed on separately.

8.4 The web server in St Andrews

Currently, the actually published version of the web site is contained in the directory `/gap/GapWWW` on the following machine in St Andrews:

```
gap-web.host.cs.st-andrews.ac.uk
```

To get access to this data the easiest and most secure way is probably to create an RSA key pair, append the public key to `/gap/.ssh/authorized_keys` and to keep the private key in the `.ssh` subdirectory of the user's home directory.

Before performing an update on `gap-web`, it is wise to check first whether jekyll runs without an error message in your own checked out version of the website.

The next step then is to test it on the online test version of the website. To do so, ssh into `gap-web` and the enter these commands:

```
cd ~/test.gap-system.org
git pull
jekyll build
```

Assuming this worked without error message, you can review the results at `https://test.gap-system.org`.

Finally, to update the actual website, use the following:

```
cd ~/www.gap-system.org
git pull
jekyll build
```

8.5 Installation on the web server

This section describes the procedure to install the GAP web site on a machine from scratch. Thus, this section is usually not needed because all this is already done on the machine `gap-web.host.cs.st-andrews.ac.uk`. However, if one wants to have an exact copy of the web site or have to install it somewhere anew, this section is needed.

8.5.1 Needed ingredients

- standard tools: `git`, `tar`, `gzip`, `make`, `sh`
- jekyll
- a web server if pages shall be published
- a copy of the full `doc` directory from a GAP installation for references into the manual (this can reside on some web site)
- setup for automatic creation of the pages for packages

8.5.2 Installation procedure

1. Clone the git repository GapWWW:

   ```
   git clone https://github.com/gap-system/GapWWW
   ```

   This creates a subdirectory `GapWWW` in the current directory.

2. Unpack some (frozen) subtrees, which are in archives:

   ```
   cd GapWWW
   gzip -dc ForumArchive.tar.gz | tar xvf -
   cd Gap3
   gzip -dc Manual3.tar.gz | tar xvf -
   cd ..
   ```

3. TODO/FIXME/WARNING: everything after this is outdated!!!

   Edit `GapWWW/lib/config`, see that file for instructions:

```
vi lib/config
```

In this file a few variables have to be defined to adapt the web pages to the local conditions.

4. Copy a whole `doc` directory of a GAP distribution to the place mentioned in `GapWWW/lib/config` (see step 4.) in the variable `GAPManualLink` (this is `GapWWW/Manuals` in the current setup).

5. The files for the GAP bibliography have been included into this directory tree in the repository.

   Create the html and PDF versions by:

   ```
   cd Doc/Bib
   gap4 convbib.g
   cd ../..
   ```

   Some more information about this is in `GapWWW/Doc/Bib/INFO` which is unchanges since 2010 and may be somewhat outdated.

6. Install package manuals:

   Copy the result of Frank's scripts to the place mentioned in `GapWWW/lib/config` (in the variable `pkgmixerpath`). (currently, this is `GapWWW/Manuals`, copy the whole `pkg` directory)

   To update the package pages, copy all `.mixer` files and `pkgconf.py` to `GapWWW/Packages` and rerun the Mixer.

7. Make sure that the file `GapWWW/lib/AllLinksOfAllHelpSections.data` is always up-to-date (this has to be adjusted whenever the released manuals change).

   In the development version of GAP there is a file `dev/LinksOfAllHelpSections.g`. Read this with a current GAP version with all currently released packages installed and call `WriteAllLinksOfAllHelpSections()`, this writes the file `AllLinksOfAllHelpSections.data`. It has then to be checked in to its place under the `GapWWW` tree. Do not forget to publish the latest revision.

8. Run Jekyll:

   ```
   jekyll build
   ```

8.5.3 Installing updated versions

If things are changed in the repository, all that has to be done to update the pages locally is:

```
git pull
```

in the `GapWWW` directory, followed by a

```
jekyll build
```

8.6 The GAP manuals on the web pages

All GAP manuals are available in HTML format via the web pages. This works by simply copying the `doc` directory of a complete GAP installation to the place specified by the variable `GAPManualLink` in `GapWWW/lib/config` (which is `GapWWW/Manuals` in the current setup). Note that those files are *not* under version control there, they are only copied to checked out working copies, like for example on the web server in St Andrews.

The single remaining point to explain is how one can specify links to manual sections on the web pages. This is done with a special Mixer tag like the following:

```
<mixer manual="Reference: Lists">Chapter about lists</mixer>
```

This element creates a link to the manual section which would appear in the GAP help system when called with "`?Reference: Lists`", which happens to be the chapter in the reference manual about lists. The text of the link would be "Chapter about lists".

This works, because the Mixer has access to a file containing the links to all manual sections. This file resides in `GapWWW/lib/AllLinksOfAllHelpSections.data`, which is created using `dev/LinksOfAllHelpSections.g` in the development version of GAP as described in Section 8.5.

The value of the attribute "manual" in the "mixer" tag must be the complete text of the section heading the link should point to.

8.7 The GAP packages on the web pages

The archives and web pages for the GAP packages are generated by yet another set of tools described in Chapter 7. These generate for every package a `.mixer`-file and for all packages together a file `pkgconf.py`. All these files have to be put under version control in the directory `GapWWW/Packages`. These nodes then only have to be put into the tree by mentioning them in the `tree` file there.

8.8 The GAP forum archive

Until December 2003 the GAP forum archive was handled by a tool written especially for this task. At that point it was switched to `mailman`, a generic tool for mailing list, which also does the archiving. Therefore the old forum archives are frozen in form of a huge amount of HTML pages. These are not kept under version control as single files but as one big binary archive under `GapWWW/ForumArchive.tar.gz`.

To install those pages in a checked out working copy one just has to extract this archive by doing

```
gzip -dc ForumArchive.tar.gz | tar xf -
```

in the `GapWWW` directory as explained in Section 8.5.

Chapter 9

Hints and conventions for writing GAP programs

Idea: Collect hints or ideas on questions like: name clashes, functions versus operations, declaration and implementation files, organising large sets of data, ...

(We certainly don't want to develop "obligatory coding guidelines", but it may be sensible to fix some unwritten conventions to reduce clashes between the library and packages and to make the use of GAP easier.)

Such a section may also be useful for editors/referees of packages.

9.1 Naming conventions

By convention, names of documented global variables are usually concatenations of capitalized words, like e.g. `ConjugacyClassesSubgroups`. Thus, words are separated by capital letters rather than underscores. Names of only internally used global variables are usually written in capital letters only, and words are separated by underscores, like e.g. `SHALLOWCOPY_GF2MAT`.

As there is only one global name space, some care has to be taken to avoid name clashes between packages or between packages and the library. A useful rule in this context is the following:

Names standing for general concepts can be introduced in the library as names of wrapper functions like e.g. `Group` or of operations or attributes. If they are introduced in packages, they should always be used for operations or attributes to allow other packages to install methods as well or to declare an operation of the same name with a different scope.

Index